Homework 3: Neural Networks on Oracle Bone Characters

Pre-Registration Report Submission Deadline: 11/5 11:59 PM
Final Report and Code Submission Deadline: 11/19 11:59 PM

Introduction

A museum is digitizing rubbings of oracle bone inscriptions (甲骨文), the oldest known Chinese characters, carved on bones and shells more than three thousand years ago. Each rubbing has been cut into small grayscale images, one character per image. The curators want a system that suggests which character an image shows, so that experts only have to confirm or correct the suggestion.

In Homework 2 you compared nearest-neighbor, Bayesian, tree-based, and support-vector classifiers on a table of hand-crafted features. This time there are no hand-crafted features: the input is the raw pixels of each image. The team asks whether a **neural network** should be used for the task. Neural networks are trained very differently from the models in Homework 2: they are fitted by gradient descent on a loss function, through layers of learned non-linear units, and what they learn depends heavily on their size, how they are optimized, and how they are regularized. Before trusting one, the team needs to answer questions such as:

- Why does a network need an activation function, and what does a hidden layer add to a model without one?
- How do the size and the depth of a network, and its other hyperparameters, change what it learns?
- Why do some networks fail to learn at all, and how do the activation function, the optimizer, the learning rate, the batch size, and the preprocessing of the inputs change what happens during training?
- When a network memorizes its training images, which regularization methods help, and what does each one change?

Your goal in this assignment is to answer these questions with evidence. The purpose of the homework is not to obtain the highest predictive accuracy. Apart from the bonus in Section 6 Question 2, you will not compete with your classmates. You are expected to connect each conclusion to evidence, explain

what you observe from how each algorithm is defined, and say clearly when the data do not support a conclusion. A weak or unsuccessful result can be a correct and important finding if you demonstrate it carefully and explain what it means.

The dataset has been provided to you. Do not add external datasets, external features, pre-trained models, or information collected from other sources. Your conclusions should be supported by evidence from your analysis of the given dataset.

Rules

1. This is an individual assignment. You may discuss course concepts and general approaches with classmates, but the analysis you perform, the code you submit, and the writing in your report must be your own. Do not copy another student's code, results, figures, or written explanations. **Any submission found to contain plagiarism or copied work will receive a score of zero for the assignment.**
2. AI tools, including coding assistants and coding agents, are permitted. If you use them, you are still responsible for understanding, checking, and being able to explain or modify everything you submit. You must also submit the AI-use log described in Section 7.
3. Write the report yourself. Do not use AI tools to generate the content of your report. We want to read your own reasoning in your own words. Plain language is fine: a short, clear explanation of what you did and why is worth more than a polished paragraph you cannot defend at the demo. Reports that read as AI-generated will be checked at the demo, and you will be asked to explain each conclusion.
4. **You may only use the models, layers, and training methods taught in class up to 11/12 (Thu).** Parts of the report that rely on later material receive no credit. You should be able to relate everything you use to the lecture slides at the demo.
5. The report may be written in either Traditional Chinese or English. There are no specific requirements for the report format.
6. Google Colab is sufficient for this assignment to be accomplished; no GPU is required. **You are highly recommended to use PyTorch for all neural**

networks in this assignment. You may also use numpy, pandas, scipy, matplotlib, and scikit-learn.

7. Submit your work on eeclass. No late submission will be accepted (except the pre-registration, see Section 2). The main submission consists of:

Submission	Deadline
A pre-registration report in the PDF format, up to 4 pages. An appendix with your 30 handwritten characters does not count toward the 4 pages. File name: `HW3_pre_report_{student_id}.pdf` Your 30 handwritten characters from Section 2 Question 5, in the format described there. File name: `HW3_handwriting_{student_id}.npz`	11/5 11:59 PM
A final report in the PDF format. File name: `HW3_final_report_{student_id}.pdf` An .ipynb notebook containing all code needed to reproduce the results reported in your submission. File name: `HW3_code_{student_id}.ipynb` The AI-use log document. File name: `HW3_ai_use_{student_id}.md` Optional (bonus, Section 6 Question 2d): your predictions for the class handwriting dataset. File name: `HW3_prediction_{student_id}.npy`	11/19 11:59 PM

Materials

The following materials for Homework 3 have been posted on eeclass:

- The starter notebook `hw3_starter.ipynb`
- The dataset `oracle_chars.npz`, described in Section 1
- The build script `build_oracle_chars.py` for reference
- The lecture slides on neural networks and deep neural networks
- The PyTorch tutorial slides

Section 1: The Data

The file `oracle_chars.npz` contains 5,200 images of 28 × 28 pixels with 3 color channels (RGB), with values from 0 (black) to 255 (white). The rubbings are grayscale, so the three channels of each image are equal; they are stored in

the same format as your handwriting in Section 2 Question 5. It has four arrays: `images` (5200 × 28 × 28 × 3), `labels` (5200 integers from 0 to 9), `class_names`, and `characters`. Each image shows one of the following 10 characters:

Label	0	1	2	3	4	5	6	7	8	9
Name	big	sun	moon	cattle	next	field	not	arrow	time	wood
Character	大	日	月	牛	翌	田	勿	矢	巳	木
Images	1,100	950	800	650	520	410	320	230	140	80

Rare characters. Throughout this assignment, the rare characters are the three smallest classes: *arrow*, *time*, and *wood*. When a question asks for the recall of the rare characters, report the value of each of the three and their average.

The images were scanned from real rubbings. Many of them contain cracks, stains, and pieces of neighboring characters, and the same character was carved in very different styles. The file was built as follows (see `build_oracle_chars.py`):

1. The images come from the Oracle-MNIST collection (Wang & Deng, *Scientific Data*, 2024). Its official training and test files were pooled, and exact duplicate images were removed.
2. Each character was randomly sampled with a fixed seed to the number of images in the table above, so that the class sizes form a long tail.
3. The grayscale image was repeated in the three channels.

Variants of each character. The same character can be written in very different ways. Figure 1 shows, for each character, a main form and two variants with their stroke order, next to four images from the dataset. Use it when you write your own characters in Section 2 Question 5: you may follow these forms, and you may also change them yourself (e.g., add, remove, bend, or move a stroke) to make your characters less certain for a network.

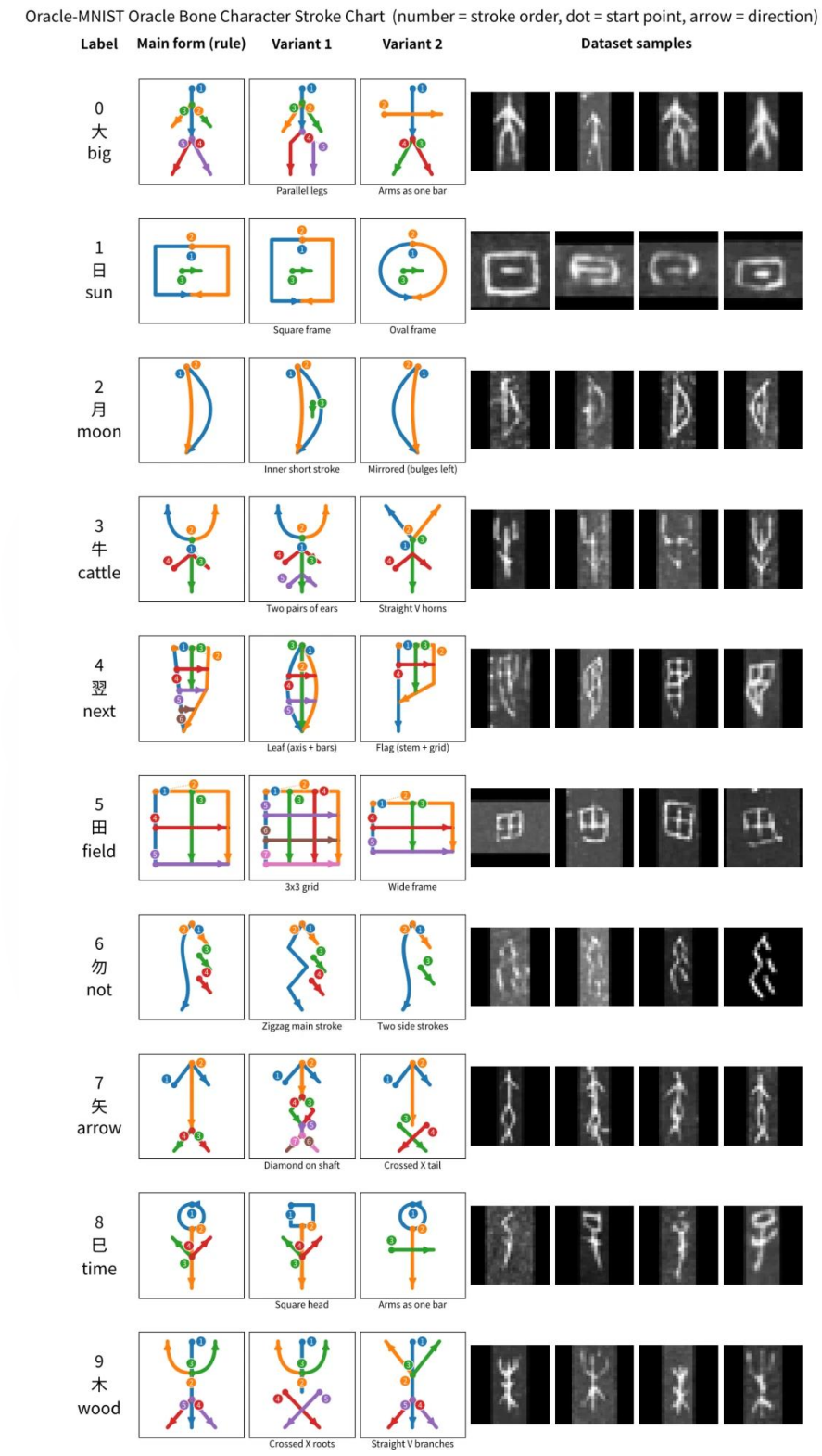


Figure 1. The 10 characters: main form, two variants, and four dataset images. The number gives the stroke order, the dot the start point, and the arrow the direction of each stroke.

Reminder. Unless a question says otherwise, the input to every model is the 2,352 values of an image (28 × 28 pixels × 3 channels), divided by 255. Do not crop, resize, or hand-engineer features from the images.

Section 2: Pre-Registration (6%)

Submit this section report before 11/5 11:59 PM. You may look at the images and count the labels, but **do not train any neural network before submitting it.** Your pre-registration is graded on completeness and specificity, not on whether your predictions are correct. You will compare these predictions with your results in Sections 3 to 6.

- Submitted by November 5: Up to 6%.
- Submitted between November 6 and November 19: Up to 4%.
- Not submitted: Section 2 receives 0 points, and Section 6 Question 2 cannot receive credit.

Only the handwriting files submitted by November 5 are included in the class handwriting dataset of Section 6 Question 2d.

In at most four pages, provide the following. The 30 handwritten characters of Question 5 may be put in an appendix that does not count toward the page limit.

1. **(1%) Explore the data.** Look at the data distribution and perform at least three different exploratory data analyses (EDA) of the images and the labels (e.g., the number of images per class, the average image of each class, the histogram of the pixel values). Show a figure for each and state what it reveals. Based on them, what problems do you expect if a network is trained directly on these images, and what preprocessing would you apply to address each problem?
2. **(1%) Activation functions.** What is the effect of using an activation function? Predict what happens when the hidden layers use no activation function (activation(x) = x), ReLU, or sigmoid: what the network can represent, how fast it trains, and how well it classifies these images. Explain why in terms of the gradient: how does the derivative of each activation function affect the gradients that backpropagation sends to the earlier layers, and the size of the weight updates there?

3. **(1%) Your two main networks.** Design two networks: a shallow network that is wide and has few hidden layers, and a deep network that is narrow and has many hidden layers. Their total numbers of trainable parameters (weights and biases) must differ by at most 2% of the smaller total. Give the number of hidden layers and the number of units in each layer, write each total as a formula of these hyperparameters, compute both numbers, and show that the difference is within 2%. **These two networks are your main networks in the rest of this assignment: whenever a later question says "your two main networks" or "the same network", it means the networks you design here, in Section 2 Question 3.**

4. **(1%) Without preprocessing.** What problems do you expect if the images are fed to a network without any preprocessing, both during training and in the predictions? What are the limitations of this approach?

5. **(2%) Write your own oracle characters.** Open the dataset and look at several images of each of the 10 characters, and the variants in Figure 1. Then write each character 3 times by hand, 30 characters in total. The 3 copies of a character must all be different: for example, write the main form and the variants of Figure 1, change them yourself to make them less certain, or change the size, the position, or the stroke thickness. Scan or photograph them and include all 30 in your report. Describe how you made the copies different, and predict how many of your 30 characters your network will classify correctly (you will classify them in Section 6 Question 2). Also submit them in one common format, so that the whole class's characters can be put together into one dataset:

   - Crop a square around each character and resize it to 28 × 28 pixels. Keep the 3 color channels (RGB) and the original colors (dark ink on light paper): do not convert to grayscale, invert, or otherwise process them.
   - Save all 30 in one file `HW3_handwriting_{student_id}.npz` with two arrays: `images`, a uint8 array of shape 30 × 28 × 28 × 3 with values from 0 to 255, and `labels`, the 30 labels in the order 0, 0, 0, 1, …, 9 (the 3 copies of each character together). The function `save_handwriting` in the starter notebook builds and checks this file from your 30 cropped images.

Section 3: Inside a Neural Network (10%)

In this and all later sections:

- Split the data once into 80% training and 20% testing, stratified by class, using your student ID as the random seed. Use your student ID as the seed everywhere else too (e.g., the weight initialization and the shuffling), and train each network once; there is no cross-validation in this assignment. Training a network also requires validation data for early stopping and for choosing hyperparameters: take it from the training set (e.g., a stratified 20% of it) and state exactly how, so that the test set never influences any decision. This is your protocol.
- Fit every preprocessing step only on the training part, and never use the test set until Section 6 Question 1.
- For each network, report its number of hidden layers, the number of units in each, the activation function, the loss, the optimizer, the learning rate, the batch size, the number of epochs, and how the weights are initialized. Library defaults are choices too: identify and justify the ones you rely on.
- When a question says "the same network", use one of your two main networks from Section 2 Question 3 and say which one. Unless a question says otherwise, use the same number of epochs for all networks compared within a question.
- Unless a question says otherwise, evaluate every network on the validation data with the accuracy, the macro-averaged precision, recall, and F1-score, and the recall of the rare characters. Write every recall also as k/n, the number of correctly classified images out of the number of images of that class (e.g., 9/13): the rare characters have only a few images, so one image can change their recall a lot.
- In every question, plot the confusion matrix of each network whose classification scores you report (not of every setting in a hyperparameter search), and refer to it in your answer.
- Explain your results from the formulas in the lecture slides, and back every claim with numbers.

1. (6%) ReLU or sigmoid?

a. Train two networks with the same two hidden layers of the same width: (i) with ReLU and (ii) with sigmoid. Report the number of parameters and the

validation accuracy, macro-F1, and rare-character recall of each, and compare them with your prediction in Section 2 Question 2.

b. The perceptron in the slides uses a step function as its activation. A step unit computes the weighted sum $z = \mathbf{w} \cdot \mathbf{x} + b$ of its inputs and outputs 1 if $z \geq 0$ and 0 if $z < 0$, so its output jumps from 0 to 1 at $z = 0$ and is flat everywhere else. Explain why a network of step units cannot be trained by backpropagation, and what the sigmoid changes.

c. Reading the scores. From the confusion matrix of network (i), compute by hand the precision, the recall, and the F1-score of one common character and one rare character, and show each step. Then explain the difference between the accuracy and the macro-F1, and between the macro and the weighted average: why can a network have a high accuracy but a low macro-F1 on this dataset? If the museum mainly wants to find the rare characters, which score should it look at, and why?

2. (4%) Choosing the loss

a. Train the same network (one of your two main networks from Section 2 Question 3) with ReLU hidden units, once with the cross-entropy loss and once with the squared-error loss between the softmax outputs and the one-hot labels, for 50 epochs each. Plot the training loss and the validation macro-F1 against the epoch for both. Using the derivative of the loss with respect to the input of an output unit, explain why one learns faster than the other, especially when an output is confidently wrong.

Section 4: Shallow and Deep Networks (8%)

1. (4%) Shallow or deep?

a. Train your two main networks from Section 2 Question 3, keeping everything else fixed. Report the number of parameters in each layer, the training and validation macro-F1, the rare-character recall, and the training time of each, and plot their training and validation loss against the epoch.

b. Analyze the comparison. Which network is better on these images? With one run of each, how sure can you be that the difference is not due to chance? The slides argue that deep networks learn hierarchical representations. Does your evidence support that argument for this dataset and these networks? If not, explain what might be missing (e.g., think

about the manifold that these images lie on). Which network would you choose for the museum, and why?

2. (4%) Vanishing gradients

a. Add batch normalization to a network with 10 hidden layers of sigmoid units; state the width of the layers and where you place batch normalization. This question needs no confusion matrix. For the network with and without batch normalization, report the norm of the gradient of the loss with respect to the weights of each layer, at initialization and after one epoch, computed on the same batch of training images, and plot the training loss of both. In PyTorch, call `model.zero_grad()` and then `loss.backward()` on that batch; the gradient of the weights of a layer `layer = nn.Linear(...)` is then the tensor `layer.weight.grad`, and its norm is `layer.weight.grad.norm().item()`. To get every layer, loop over `model.modules()` and keep the modules that are `nn.Linear` (the batch-normalization layers have weights too). Explain the result from the derivative of the sigmoid and the chain rule: what changes, and why?

Section 5: Training a Network (13%)

1. (5%) Large-batch and small-batch training

a. Train the same network (one of your two main networks from Section 2 Question 3) with plain SGD (no momentum), the same learning rate, and the same number of epochs, using two batch sizes: a small one (e.g., 8) and a large one (e.g., 512). Plot the training loss against (i) the epoch, (ii) the number of weight updates, and (iii) the wall-clock time. Explain the differences from how many updates each batch size makes per epoch, how much computation each update costs, and how noisy each gradient estimate is.

b. Report the training and validation macro-F1 of each batch size. Is there a difference in generalization between small and large batches? Give a possible explanation and say how strong your evidence is.

2. (3%) Optimizers

a. Choose one learning rate and keep it fixed. Train the same network (one of your two main networks from Section 2 Question 3) with plain SGD (no momentum) and with one other optimizer from the slides of your choice

(e.g., SGD with momentum, AdaGrad, RMSProp, or Adam): two runs in total, for 50 epochs each. Plot the training loss and the validation macro-F1 against the epoch, and justify the learning rate you chose.

b. Which optimizer is better here, and why? Write the update rule of each optimizer, explain your answer from them, and check whether your experiment in (a) agrees.

3. (5%) With or without preprocessing

a. Train the same network (one of your two main networks from Section 2 Question 3) on the images without any preprocessing (the raw pixel values from 0 to 255) and with your own preprocessing. Your preprocessing may include any of the steps you proposed in Section 2 Question 1 (e.g., scaling, standardization, denoising, centering the character); describe each step. Plot the training loss and the validation macro-F1 against the epoch, and report the validation macro-F1 and the rare-character recall of both.

b. Explain the difference from the gradient descent update and the activation function: how the scale and the mean of the inputs change the weighted sums of the first hidden layer, the saturation of sigmoid units (if your network uses ReLU, explain what would happen to sigmoid units instead), the gradients of the first-layer weights, and the learning rate that works.

c. Compare the results with your answers to Section 2 Questions 1 and 4. Which problems did preprocessing solve, and which remain? Explain why the statistics of any preprocessing step must be computed on the training part only, and what goes wrong if they are not.

Section 6: Regularization, Tuning, and Your Own Handwriting (13%)

1. (5%) Tuning the regularization penalty

a. For the network you chose in Section 4 Question 1b (one of your two main networks from Section 2 Question 3), keep the learning rate fixed and tune the strength of the L2 penalty (the weight decay) in any way you like, with the validation data defined in your protocol. Report the learning rate, the search space, the number of settings tried, the chosen setting, and its validation macro-F1. Plot the validation macro-F1 against the penalty and explain its shape: what happens when the penalty is too small or too large?

b. Train the chosen setting on the whole training set, for the number of epochs chosen on the validation data, and evaluate it once on the test set. Report the accuracy, the macro-F1, the recall of each rare character, and the confusion matrix, and compare them with the validation results.

2. (8%) Your own handwriting

a. Convert the 30 characters in the file you submitted with your pre-registration (Section 2 Question 5) into the same format as the dataset: 28 × 28 × 3 images with values from 0 to 255, and a bright character on a dark background. Both your images and the dataset images have 3 color channels, so you do not need to turn them into one. Describe every step of your conversion. Show your images before and after it next to a few dataset images of the same characters. You may choose your own conversion, but apply one rule in the same way to all 30 images, followed by the same preprocessing that your network was trained with; do not adjust individual images.

b. Classify your 30 images with the network you trained in Question 1b, without retraining it on them. Report the accuracy, the number of correct copies of each character, and the confusion matrix, and compare them with the test set results and with your prediction in Section 2 Question 5.

c. Why is the network accurate or inaccurate on your handwriting? Compare your images with the dataset images (e.g., the background, the noise, the stroke thickness, the size and the position of the character, the average image of each class), and relate your answer to how your network uses each pixel. Discuss the color channels too: the three channels of a dataset image are equal, but those of your images are not (e.g., the color of the ink and the paper, the lighting). How does your network treat a difference between the channels, and how could it change the accuracy? Test at least one of your explanations with an experiment, for example by changing one step of your conversion, and report whether the accuracy changes as you expected.

d. **(Bonus 5%) Predicting the class handwriting.** After the pre-registration deadline, the TAs will combine the handwriting files of the whole class into one dataset, shuffle the images, remove their labels and writers, and post it on eeclass in the same format as your handwriting file. Classify every image with your network from Question 1b and the same conversion as in (a), and submit the predictions as

`HW3_prediction_{student_id}.npy`: an integer array with one predicted label (0 to 9) for each image, in the order of the posted file. The predictions must come from the code in your notebook: do not label images by hand, and do not train on the class dataset. The TAs will compute the accuracy of every submission on the hidden labels, and the 10 submissions with the highest accuracy (including ties at 10th place) receive 5%.

Section 7: AI-Use Log

AI tools are permitted in this assignment. The purpose of this log is to document how you used them and how you verified their outputs. Submit an AI-use log written as a markdown file. If you used any AI tool during the assignment, briefly describe:

1. **What you used AI for.** List the main parts of the assignment for which you used AI, such as writing or debugging code, explaining a concept, suggesting an analysis, or interpreting a result.
2. **How you checked the output.** For each major use, briefly explain how you verified that the AI-generated output was correct or appropriate. For example, you might compare it with the lecture material, inspect the code, run an additional experiment, or check the result against the data.

If you did not use any AI tools, simply state that in the log. You remain responsible for all code, analyses, figures, and explanations in your submission. You should be able to explain any AI-assisted work during the demo session. A 5-point deduction will be applied for missing submissions or conspicuously low-effort responses (e.g., merely writing "I used ChatGPT").

Grading Policy

Report (50%)

Tier	Description	Percentage
4	Provides evidence, explanation, and decisions, along with a discussion of limitations or alternative interpretations.	100%
3	Provides evidence, explanation, and decisions, with coherent logic linking all three.	80%
2	Presents results or figures, but the explanation is weak or disconnected from the evidence.	50%
1	Only presents results without explanation, or the methodology is clearly flawed.	20%
0	No answer provided / Blank.	0%

Demo (50%)

The HW3 demo will be held on 11/26. Further details, including the exact time, location, and demo format, will be announced later.

Feel free to ask questions! For homework-related issues, please post on the eeclass discussion forum, and the TAs will reply promptly.

Since you made it to the end, here are two jokes.

Joke 1. While I was an undergraduate, I discovered the Fundamental Theorem of Demo:

$$\int \mathrm{d}emo = emo + C$$

where C is your remaining sanity, which is usually 0. Every TA has verified this theorem experimentally.

Joke 2.



Ha, you still have projects, homework, exams, midterms, and finals.

Thank you for reading these bad jokes, this is the reference answer, there may be some mistakes.